

UNICID – UNIVERSIDADE CIDADE DE SÃO PAULO

CIÊNCIA DA COMPUTAÇÃO

AVALIAÇÃO A2 — PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Jantar dos Filósofos em Go — Tema C2 (Paradigma Concorrente)

Grupo 02

SÃO PAULO – SP

2026

Gabriel Coutinho Silva
João Victor Gomes Meira
Rayssa Bianca de Oliveira
Vinicius Augusto Rodrigues Silva

AVALIAÇÃO A2 — PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Jantar dos Filósofos em Go — Tema C2 (Paradigma Concorrente)

Trabalho acadêmico (Avaliação A2) apresentado à disciplina de Paradigmas de Linguagens de Programação, do curso de Ciência da Computação, como requisito parcial de avaliação, sob orientação do(a) professor(a) Cid Rodrigues de Andrade.

SUMÁRIO

1	Integrantes (3 a 5 alunos)	1
2	Definição do Tema	1
3	Execução do Código	1
3.1	Instruções de Teste (Entrada e Saída)	1
4	Relatório Técnico	5
4.1	Descrição da Solução	5
4.2	Análise Comparativa	5
4.3	Sintaxe	6
4.4	Semântica	6
4.5	Gerenciamento de Memória	7
4.6	Trade-offs	8
4.7	Resumo da Análise Comparativa	8
5	Log de Uso de Inteligência Artificial (IA)	9
	REFERÊNCIAS	10

Este arquivo é o esqueleto do **README.md** do repositório GitHub do seu grupo. A Entrega deve ser feita exclusivamente pelo envio do link desse repositório. Quaisquer instruções anteriores em desacordo com esta devem ser ignoradas. Preencha cada seção abaixo. Não altere a ordem e nem apague os títulos — apenas substitua os placeholders pelo conteúdo do seu projeto.

Grupo 02

1 INTEGRANTES (3 A 5 ALUNOS)

- Gabriel Coutinho Silva (Matrícula)
- João Victor Gomes Meira (Matrícula)
- Rayssa Bianca de Oliveira (Matrícula)
- Vinicius Augusto Rodrigues Silva (Matrícula)

2 DEFINIÇÃO DO TEMA

- **Paradigma:** Concorrente
- **Linguagem:** Go
- **Código do Tema:** C2
- **Descrição do Desafio:** Implementar o problema clássico de sincronização dos N filósofos disputando garfos/hashis compartilhados, evitando deadlock e starvation, utilizando os mecanismos de concorrência do Go. O foco do trabalho é comparar as estratégias de sincronização disponíveis em Go (`channels`, `sync.Mutex`, `sync.WaitGroup`) com os mecanismos de sincronização já vistos em outras linguagens (como `threads` e `locks` em Python).

3 EXECUÇÃO DO CÓDIGO

Aviso: O código deve rodar exclusivamente em ambiente online, sem necessidade de instalação local.

- **Ambiente Online Utilizado:** Go Playground (go.dev/play)
- **Permalink:** https://go.dev/play/p/MFBO_ue01rr

3.1 Instruções de Teste (Entrada e Saída)

Explique de forma clara como executar o código no ambiente online linkado.

O programa não recebe nenhuma entrada de dados do usuário (não há leitura de `stdin`): toda a simulação é autocontida. Para testar, basta acessar o Go Playground, colar o código-fonte abaixo (idêntico ao arquivo `filosofos.go` deste repositório) e clicar em **Run**. A simulação inicia automaticamente e o resultado é impresso diretamente no console de saída do Playground.

Entrada de Exemplo:

```

package main

import (
    "fmt"
    "math/rand"
    "sync"
    "time"
)

// =====
// Parâmetros da simulação
// =====
const (
    numPhilosophers      = 5 // N filósofos e N garfos
    mealsPerPhilosopher = 3 // quantas vezes cada filósofo deve comer
)

// Fork representa um garfo compartilhado. Cada garfo é protegido por seu
// próprio Mutex: apenas um filósofo pode segurá-lo por vez.
type Fork struct {
    mu sync.Mutex
    id int
}

// philosopher é a goroutine que representa o ciclo de vida de UM filósofo:
// pensar -> tentar pegar os garfos -> comer -> devolver os garfos -> repetir.
func philosopher(id int, leftFork, rightFork *Fork, seating chan struct{}, wg *sync.WaitGroup) {
    defer wg.Done()

    for meal := 1; meal <= mealsPerPhilosopher; meal++ {
        think(id)

        // (1) Pede permissão ao "garçom" (arbitrador de Dijkstra) antes de
        // disputar os garfos. O canal `seating` tem capacidade N-1, ou seja,
        // no máximo N-1 filósofos podem estar tentando pegar garfos ao mesmo
        // tempo. Isso torna IMPOSSÍVEL que os N filósofos fiquem, cada um,
        // com um garfo na mão esperando pelo vizinho - a condição necessária
        // para deadlock (espera circular) nunca se forma.
        seating <- struct{}{}

        // (2) Aquisição ORDENADA dos garfos: sempre trava primeiro o garfo
        // de menor id. Essa é uma segunda barreira independente contra
        // deadlock (quebra de espera circular por ordenação total de
        // recursos), mantida mesmo que a capacidade do "garçom" mude no
        // futuro.
        first, second := leftFork, rightFork
        if first.id > second.id {
            first, second = second, first
        }

        first.mu.Lock()
        second.mu.Lock()

        fmt.Printf("Filósofo %d pegou os garfos %d e %d e está COMENDO (refeição %d)\n",
            id, first.id, second.id, meal)
        eat(id)

        second.mu.Unlock()
    }
}

```

```

    first.mu.Unlock()

    // (3) Libera o assento para que outro filósofo possa tentar comer.
    <-seating

    fmt.Printf("Filósofo %d devolveu os garfos %d e %d (refeição %d concluída)\n",
        id, leftFork.id, rightFork.id, meal)
}

fmt.Printf(">>> Filósofo %d terminou todas as suas refeições e saiu da mesa.\n", id)
}

func think(id int) {
    fmt.Printf("Filósofo %d está PENSANDO...\n", id)
    time.Sleep(time.Duration(rand.Intn(80)+40) * time.Millisecond)
}

func eat(id int) {
    time.Sleep(time.Duration(rand.Intn(80)+40) * time.Millisecond)
}

func main() {
    rand.Seed(time.Now().UnixNano())

    // Cada garfo é um recurso compartilhado independente, identificado por
    // um índice de 0 a N-1.
    forks := make([]*Fork, numPhilosophers)
    for i := 0; i < numPhilosophers; i++ {
        forks[i] = &Fork{id: i}
    }

    // Canal usado como semáforo contador (não para trocar dados, e sim como
    // mecanismo de sincronização): representa o "garçom" do problema
    // clássico. Capacidade N-1 garante que ao menos um filósofo sempre
    // consiga liberar um garfo, prevenindo deadlock.
    seating := make(chan struct{}, numPhilosophers-1)

    var wg sync.WaitGroup
    wg.Add(numPhilosophers)

    // Cada filósofo é uma goroutine independente, concorrendo pelos dois
    // garfos vizinhos (esquerdo e direito) dispostos em círculo.
    for i := 0; i < numPhilosophers; i++ {
        left := forks[i]
        right := forks[(i+1)%numPhilosophers]
        go philosopher(i, left, right, seating, &wg)
    }

    // A goroutine principal aguarda todas as goroutines dos filósofos
    // terminarem antes de encerrar o programa.
    wg.Wait()
    fmt.Println("Todos os filósofos terminaram. Simulação encerrada com sucesso.")
}

```

Saída Esperada:

```

Filósofo 4 está PENSANDO...
Filósofo 0 está PENSANDO...
Filósofo 1 está PENSANDO...
Filósofo 3 está PENSANDO...

```

```

Filósofo 2 está PENSANDO...
Filósofo 0 pegou os garfos 0 e 1 e está COMENDO (refeição 1)
Filósofo 2 pegou os garfos 2 e 3 e está COMENDO (refeição 1)
Filósofo 0 devolveu os garfos 0 e 1 (refeição 1 concluída)
Filósofo 0 está PENSANDO...
Filósofo 4 pegou os garfos 0 e 4 e está COMENDO (refeição 1)
Filósofo 2 devolveu os garfos 2 e 3 (refeição 1 concluída)
Filósofo 2 está PENSANDO...
Filósofo 1 pegou os garfos 1 e 2 e está COMENDO (refeição 1)
Filósofo 4 devolveu os garfos 4 e 0 (refeição 1 concluída)
Filósofo 4 está PENSANDO...
Filósofo 3 pegou os garfos 3 e 4 e está COMENDO (refeição 1)
Filósofo 1 devolveu os garfos 1 e 2 (refeição 1 concluída)
Filósofo 1 está PENSANDO...
Filósofo 0 pegou os garfos 0 e 1 e está COMENDO (refeição 2)
Filósofo 3 devolveu os garfos 3 e 4 (refeição 1 concluída)
Filósofo 3 está PENSANDO...
Filósofo 2 pegou os garfos 2 e 3 e está COMENDO (refeição 2)
Filósofo 0 devolveu os garfos 0 e 1 (refeição 2 concluída)
Filósofo 0 está PENSANDO...
Filósofo 4 pegou os garfos 0 e 4 e está COMENDO (refeição 2)
Filósofo 4 devolveu os garfos 4 e 0 (refeição 2 concluída)
Filósofo 4 está PENSANDO...
Filósofo 2 devolveu os garfos 2 e 3 (refeição 2 concluída)
Filósofo 2 está PENSANDO...
Filósofo 3 pegou os garfos 3 e 4 e está COMENDO (refeição 2)
Filósofo 1 pegou os garfos 1 e 2 e está COMENDO (refeição 2)
Filósofo 1 devolveu os garfos 1 e 2 (refeição 2 concluída)
Filósofo 1 está PENSANDO...
Filósofo 0 pegou os garfos 0 e 1 e está COMENDO (refeição 3)
Filósofo 0 devolveu os garfos 0 e 1 (refeição 3 concluída)
>>> Filósofo 0 terminou todas as suas refeições e saiu da mesa.
Filósofo 3 devolveu os garfos 3 e 4 (refeição 2 concluída)
Filósofo 3 está PENSANDO...
Filósofo 4 pegou os garfos 0 e 4 e está COMENDO (refeição 3)
Filósofo 2 pegou os garfos 2 e 3 e está COMENDO (refeição 3)
Filósofo 4 devolveu os garfos 4 e 0 (refeição 3 concluída)
>>> Filósofo 4 terminou todas as suas refeições e saiu da mesa.
Filósofo 2 devolveu os garfos 2 e 3 (refeição 3 concluída)
>>> Filósofo 2 terminou todas as suas refeições e saiu da mesa.
Filósofo 3 pegou os garfos 3 e 4 e está COMENDO (refeição 3)
Filósofo 1 pegou os garfos 1 e 2 e está COMENDO (refeição 3)
Filósofo 3 devolveu os garfos 3 e 4 (refeição 3 concluída)
>>> Filósofo 3 terminou todas as suas refeições e saiu da mesa.
Filósofo 1 devolveu os garfos 1 e 2 (refeição 3 concluída)
>>> Filósofo 1 terminou todas as suas refeições e saiu da mesa.
Todos os filósofos terminaram. Simulação encerrada com sucesso.

```

Observação: como a ordem exata de execução das goroutines depende do escalonador do Go e de tempos de espera aleatórios (`rand.Intn`), a saída acima é **um exemplo real de execução**, não uma saída fixa e determinística — a ordem das linhas pode variar levemente a cada execução, mas o padrão geral (pensar → comer → devolver garfos → repetir 3 vezes → sair da mesa) e a ausência de deadlock/starvation se mantêm sempre.

4 RELATÓRIO TÉCNICO

4.1 Descrição da Solução

O grupo implementou o problema clássico do **Jantar dos Filósofos** com $N = 5$ filósofos e $N = 5$ garfos dispostos em uma mesa circular, onde cada garfo é compartilhado entre dois filósofos vizinhos. Cada filósofo precisa dos dois garfos ao seu lado para comer, e o desafio central é sincronizar o acesso a esse recurso compartilhado sem que o sistema trave (deadlock) e sem que um filósofo específico fique impedido de comer indefinidamente (starvation).

Arquitetura da solução:

- **Fork (struct):** representa um garfo. Contém um `id` (para identificação e para a estratégia de ordenação) e um `sync.Mutex` próprio, que garante que apenas um filósofo por vez consiga “segurar” aquele garfo.
- **philosopher (goroutine):** é a função que representa o ciclo de vida completo de um filósofo — pensar -> pedir permissão -> travar os dois garfos -> comer -> destravar os garfos -> devolver a permissão -> repetir. Cada um dos 5 filósofos é executado como uma **goroutine independente**, disparada com a palavra-chave `go` a partir da função `main`, o que faz com que todos os filósofos “existam” e ajam concorrentemente, e não em sequência.
- **seating (channel bufferizado, capacidade N-1):** funciona como o “garçom”/arbitrador de Dijkstra. Antes de tentar pegar os garfos, cada filósofo precisa ocupar uma “vaga” nesse canal; como só existem $N-1$ vagas para N filósofos, nunca é possível que todos os filósofos estejam, ao mesmo tempo, com um garfo na mão disputando o segundo — o que é a primeira barreira contra deadlock.
- **Aquisição ordenada dos garfos:** além do arbitrador, cada filósofo sempre trava primeiro o garfo de **menor id**, independentemente de qual seja seu garfo “esquerdo” ou “direito”. Essa ordenação total elimina a possibilidade de um ciclo de espera circular — a segunda barreira, independente da primeira, contra deadlock.
- **sync.WaitGroup:** utilizado pela função `main` para saber quando todos os 5 filósofos terminaram suas `mealsPerPhilosopher(3)` refeições. A `main` chama `wg.Add(5)` antes de disparar as goroutines e bloqueia em `wg.Wait()` até que cada filósofo chame `wg.Done()` (via `defer`, garantindo que isso aconteça mesmo se algo interromper o fluxo normal).

O programa imprime, ao longo da execução, o estado de cada filósofo (pensando, comendo, devolvendo os garfos), o que torna a concorrência observável diretamente no console — é possível ver mensagens de filósofos diferentes intercaladas, evidenciando que eles realmente executam ao mesmo tempo.

4.2 Análise Comparativa

Compare profundamente a linguagem e o paradigma estudados com os que você já domina (C, Java ou Python). Desenvolva suas respostas nos tópicos abaixo, conectando a prática com a

teoria vista na disciplina.

4.3 Sintaxe

A sintaxe de Go é enxuta e explícita, especialmente no que diz respeito à concorrência, que é tratada como um recurso de primeira classe da linguagem (e não uma biblioteca externa). Para comparar, o grupo também implementou o mesmo problema em Python (arquivo `filosofos.py`), usando o módulo `threading`.

Disparar uma unidade de execução concorrente para cada filósofo:

```
// GO
go philosopher(i, left, right, seating, &wg)
```

```
# PYTHON
t = threading.Thread(target=philosopher, args=(i, left, right, seating))
t.start()
```

Em Go, a concorrência é ativada com uma única palavra-chave (`go`) na frente da chamada de função — não é preciso instanciar um objeto “Thread” e chamar um método `.start()` separado, como em Python. Isso torna o código de Go mais direto quando o objetivo é apenas “rodar isso em paralelo”.

Outra diferença sintática relevante é o uso de canais (`channels`), que não têm equivalente sintático direto em Python:

```
// GO
seating <- struct{}{} // envia/ocupa uma vaga
<-seating             // recebe/libera uma vaga
```

```
# PYTHON
seating.acquire() # equivalente funcional, via threading.Semaphore
seating.release()
```

Em Python, o mesmo papel (limitar a concorrência simultânea) é desempenhado por uma classe pronta da biblioteca padrão (`threading.Semaphore`), com métodos nomeados. Em Go, o mesmo efeito é obtido usando um recurso da própria sintaxe da linguagem (o operador `<-` sobre um `chan` bufferizado) — não existe uma classe `Semaphore` pronta na biblioteca padrão de Go; a comunidade usa `channels` para isso.

De modo geral, o código em Go não é mais curto que o equivalente em Python, mas é mais explícito: por exemplo, o uso de ponteiros (`*Fork, &wg`) deixa claro, na própria assinatura das funções, quais dados são compartilhados por referência e quais são copiados — algo que em Python fica implícito (todo objeto é sempre referenciado, nunca copiado automaticamente).

4.4 Semântica

Tipagem: Go é uma linguagem **estaticamente tipada** e **compilada**: todo erro de tipo (por exemplo, tentar somar uma `string` com um `int`) é detectado **antes** da execução, durante a compilação. Python é **dinamicamente tipada**: o mesmo erro só apareceria **durante** a execução, no momento

exato em que a operação incorreta fosse tentada. Isso tem impacto direto em código concorrente: em Go, o compilador já garante, por exemplo, que `leftFork` e `rightFork` são sempre do tipo `*Fork`, reduzindo uma classe inteira de erros que só apareceriam em tempo de execução em Python.

Escopo: ambas as linguagens usam escopo léxico (baseado em blocos), mas Go é mais restritivo — variáveis declaradas dentro de um bloco (`{}`, como o corpo de um `for` ou de um `if`) não existem fora dele, e o compilador rejeita variáveis declaradas e nunca usadas, o que força um código mais limpo.

Forma de avaliação e comunicação: o conceito semântico mais importante deste trabalho é a diferença entre comunicação **por memória compartilhada com locks** e comunicação **por passagem de mensagens (channels)**. O código usa os dois modelos lado a lado, de forma proposital:

- Os garfos (`sync.Mutex`) representam o modelo tradicional de **memória compartilhada protegida por lock**, o mesmo modelo usado em Python com `threading.Lock`: múltiplas goroutines/threads enxergam o mesmo dado na memória, e o lock decide quem pode acessá-lo por vez.
- O canal `seating` representa o modelo idiomático de Go de **comunicação por canais**, resumido no lema da comunidade Go: “*não compartilhe memória para se comunicar; comunique-se para compartilhar memória*”. Em vez de uma goroutine olhar diretamente para uma variável compartilhada, ela envia/recebe sinais por um canal — e é o próprio canal (não um lock manual) que garante a exclusão mútua da comunicação.

4.5 Gerenciamento de Memória

Go usa **coletor de lixo (garbage collector) automático**, assim como Python — o programador não chama `free()` manualmente. Porém, o funcionamento interno é bem diferente:

- **C** exige gerenciamento **manual** de memória (`malloc/free`); esquecer de liberar memória causa vazamentos, e liberar duas vezes ou usar memória já liberada causa comportamento indefinido — problemas que simplesmente não existem em Go ou Python.
- **Python** gerencia memória principalmente por **contagem de referências** (cada objeto sabe quantas variáveis apontam para ele; quando chega a zero, é liberado), complementada por um coletor de ciclos para referências circulares. Isso tem custo de desempenho a cada atribuição/desalocação de referência.
- **Go** usa um **garbage collector concorrente de rastreamento (tracing GC)**, que roda em segundo plano e não depende de contagem de referências. Além disso, Go decide automaticamente, através de **escape analysis**, se uma variável pode viver na pilha (stack) — mais rápida e sem envolver o GC — ou precisa ir para o heap (quando, por exemplo, um ponteiro para ela “escapa” da função, como quando fazemos `forks[i] = &Fork{id: i}` e o ponteiro passa a ser usado por várias goroutines).
- Especificamente sobre concorrência: **goroutines** têm uma pilha inicial pequena (poucos KB) que cresce dinamicamente conforme necessário, o que é o que permite ter milhares

delas sem esgotar a memória — diferente de threads do sistema operacional (usadas por Python via `threading`), que reservam uma quantidade de memória fixa e bem maior por thread, tornando-as mais “caras” de criar em grande número.

4.6 Trade-offs

Vantagens do modelo concorrente de Go para este problema:

- Goroutines são muito mais leves que threads de sistema operacional, permitindo escalar a simulação para um número bem maior de filósofos sem grande custo de memória.
- O paralelismo em Go é **real** em múltiplos núcleos de CPU, sem a limitação do GIL (Global Interpreter Lock) que existe no CPython — em Python, threads não executam bytecode Python em paralelo verdadeiro na CPU (embora, neste problema específico, dominado por `time.sleep`, isso tenha pouco impacto prático, já que as threads passam a maior parte do tempo bloqueadas/dormindo, não competindo por CPU).
- Channels tornam certas formas de sincronização (como o “garçom”/semáforo) mais legíveis e integradas à sintaxe da linguagem, reduzindo a chance de erros de uso incorreto de uma API externa.

Desvantagens/limitações:

- O modelo de memória compartilhada com Mutex (usado nos garfos) ainda exige disciplina do programador — esquecer um `Unlock()`, ou travar dois Mutexes em ordens diferentes em partes diferentes do código, pode reintroduzir deadlock; Go não impede isso automaticamente (diferente de linguagens com modelos de concorrência baseados exclusivamente em passagem de mensagens sem memória compartilhada, como Elixir/Erlang).
- Comparado ao paradigma puramente sequencial (Imperativo clássico), a solução concorrente é mais difícil de raciocinar e depurar: erros de concorrência (como condições de corrida) muitas vezes só aparecem esporadicamente, dependendo do escalonamento, e são mais difíceis de reproduzir do que um bug determinístico de um programa sequencial.
- Em relação ao paradigma Orientado a Objetos “clássico” (como em Java, com `synchronized` e classes `Thread`), Go tende a produzir um código mais enxuto para o mesmo problema, mas abre mão de alguns recursos de encapsulamento típicos de OO (por exemplo, `struct` não tem modificadores de acesso por campo como `private/public` no sentido de Java — a convenção em Go é usar a inicial maiúscula/minúscula do identificador para indicar visibilidade entre pacotes).

4.7 Resumo da Análise Comparativa

Compare a linguagem/paradigma estudado com os que você já domina (C, Java ou Python). O detalhamento completo deve estar no PDF, mas inclua os pontos principais aqui.

- **Sintaxe:** Go ativa concorrência com uma única palavra-chave (`go func()`), sem precisar instanciar um objeto de thread como em Python (`threading.Thread(...).start()`); usa channels (`chan, <-`) como recurso sintático nativo para sincronização, algo que Python resolve com classes prontas da biblioteca padrão (`Lock, Semaphore`).
- **Semântica:** Go é estaticamente tipada e compilada (erros de tipo pegos antes de rodar); Python é dinamicamente tipada (erros só aparecem em tempo de execução). O código combina dois modelos de sincronização: memória compartilhada com lock (`sync.Mutex`, equivalente a `threading.Lock` do Python) e comunicação por canais (`chan`, sem equivalente direto em Python).
- **Gerenciamento de Memória:** ambas as linguagens têm coleta de lixo automática (diferente do C, que exige `malloc/free` manual); Go usa um GC de rastreamento concorrente com goroutines de pilha pequena e crescente, enquanto Python usa contagem de referências mais coletor de ciclos, e threads do SO mais “pesadas” que goroutines.
- **Trade-offs:** o modelo de Go permite escalar para muito mais unidades concorrentes e oferece paralelismo real de CPU (sem GIL), mas ainda exige disciplina manual com Mutexes; em troca de um código um pouco mais explícito que o de Python, ganha-se em clareza sobre o que é compartilhado (via ponteiros) e em segurança de tipos em tempo de compilação.

5 LOG DE USO DE INTELIGÊNCIA ARTIFICIAL (IA)

O uso de IA (ChatGPT, Claude, Gemini, etc.) é permitido e incentivado para aprendizado, mas deve ser documentado. Preencha o log abaixo:

- **O que foi pedido à IA:** [Descreva os prompts ou dúvidas enviadas]
- **Qual IA foi utilizada:** [Descreva modelo e versão. Exemplo: ‘Anthropic Claude Fable 5.1’]
- **O que foi aproveitado:** [Descreva quais partes de código, lógicas ou explicações foram utilizadas]
- **O que foi reescrito/entendido pelo grupo:** [Como o grupo adaptou a resposta da IA e o que aprenderam com isso. Lembre-se: qualquer integrante pode ser questionado no seminário.]

REFERÊNCIAS

DIJKSTRA, Edsger W. **Cooperating sequential processes**. Eindhoven: Technological University Eindhoven, 1965.

GO TEAM. **The Go Programming Language Specification**. [S. l.]: Google, [s. d.]. Disponível em: <https://go.dev/ref/spec>. Acesso em: 22 set. 2026.

GO TEAM. **Package sync**. [S. l.]: Google, [s. d.]. Disponível em: <https://pkg.go.dev/sync>. Acesso em: 22 set. 2026.

PYTHON SOFTWARE FOUNDATION. **threading — Thread-based parallelism**. [S. l.: s. n.], [s. d.]. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 22 set. 2026.